

Task 7: Conditional Logic & Decision Making (if-else)

Track: Data Science | Level 1 - Day 7 | Tools: Python, Jupyter Notebook

Level 1 - Day 7: Conditional Logic & Decision Making (`if-else`)

Track: Data Science

Task: Task 7 - Advanced Conditional Logic for Data Preprocessing and Validation

Objective: Master standard, nested, and chained conditional statements (`if`, `elif`, `else`) across data quality auditing, missing value imputation, out-of-bounds anomaly detection, and classification workflows.

1. Syntax & Core Operators Reference Guide

Concept / Keyword	Syntax	Typical Role in Data Science Pipelines
`if`	`if condition:`	Entry validation, schema assertion, null checks
`elif`	`elif condition:`	Multi-bracket binning (e.g., age or income brackets)
`else`	`else:`	Catch-all fallback, anomaly tagging, default imputations
Ternary Operator	`val_true if cond else val_false`	Quick feature engineering, binary target creation
Logical Operators	`and`, `or`, `not`	Compound threshold filtering and outlier isolation

2. Foundational Conditional Examples (8 Distinct Patterns)

In [code]

```
# Example 1: Basic if-else for Threshold Validation
# Check if model test accuracy meets production deployment criteria
accuracy_score = 0.88
threshold = 0.85

if accuracy_score >= threshold:
    deployment_status = "Ready for Staging Deployment"
else:
    deployment_status = "Model Retraining Required"

print(f"Ex 1 - Accuracy: {accuracy_score * 100}% -> Decision: {deployment_status}")
```

In [code]

```
# Example 2: Multi-condition elif Chain (Categorical Binning)
# Bin customer lifetime value (LTV) into tiers
customer_ltv = 7400

if customer_ltv >= 10000:
    tier = "Platinum Priority"
elif customer_ltv >= 5000:
    tier = "Gold Tier"
elif customer_ltv >= 2000:
    tier = "Silver Tier"
else:
    tier = "Standard / Bronze"

print(f"Ex 2 - Customer LTV ${customer_ltv:,} categorized into: {tier}")
```

In [code]

```
# Example 3: Logical 'and' Operator for Joint Constraints
# Credit limit increase eligibility (Income >= 50k AND Credit Score >= 720)
annual_income = 58000
credit_score = 745

if annual_income >= 50000 and credit_score >= 720:
    limit_action = "Automatic Credit Limit Increase Approved"
else:
    limit_action = "Standard Review / Ineligible"

print(f"Ex 3 - Credit Check: {limit_action}")
```

In [code]

```
# Example 4: Logical 'or' Operator for Flexible Triggers
# Fraud alert trigger: Transaction > $5,000 OR International Origin
transaction_amount = 6200.00
is_international = False

if transaction_amount > 5000 or is_international:
    risk_flag = "High Risk: Hold for 2-Factor Confirmation"
else:
    risk_flag = "Normal Risk: Auto-Processed"

print(f"Ex 4 - Fraud System: {risk_flag}")
```

In [code]

```
# Example 5: Logical 'not' and Null/Empty Object Auditing
# Detecting missing critical parameters in incoming payload
user_email = ""

if not user_email.strip():
    validation_error = "Missing mandatory attribute: 'user_email' is blank."
else:
    validation_error = "None (Valid)"

print(f"Ex 5 - Input Audit Result: {validation_error}")
```

In [code]

```
# Example 6: Nested Conditionals (Hierarchical Checks)
# Student scholarship: Enrollment status check before GPA review
enrolled_status = True
cumulative_gpa = 3.85

if enrolled_status:
    if cumulative_gpa >= 3.8:
        scholarship_verdict = "Full Dean's Merit Fellowship"
    elif cumulative_gpa >= 3.2:
        scholarship_verdict = "Partial Departmental Fellowship"
    else:
        scholarship_verdict = "Standard Tuition Rate"
else:
    scholarship_verdict = "Ineligible: Inactive Student Status"

print(f"Ex 6 - Academic Grant Decision: {scholarship_verdict}")
```

In [code]

```
# Example 7: Chained Comparison Range Verification
# Environmental sensor data sanity bounds (10°C to 40°C)
sensor_reading_celsius = 26.4

if 10.0 <= sensor_reading_celsius <= 40.0:
    sensor_status = "Sensor Reading Valid and Within Normal Range"
elif sensor_reading_celsius < 10.0:
    sensor_status = "Alert: Sub-Zero or Cold Spike Anomaly"
else:
    sensor_status = "Alert: Thermal Overheating Anomaly"

print(f"Ex 7 - Sensor Check ({sensor_reading_celsius}°C): {sensor_status}")
```

In [code]

```
# Example 8: Membership with 'in' and Ternary 'if-else'
# Regional shipping fee calculator using inline ternary operator
delivery_state = "California"
free_shipping_zones = ["California", "Oregon", "Washington", "Nevada"]

shipping_fee = 0.0 if delivery_state in free_shipping_zones else 14.99
print(f"Ex 8 - Shipping Destination '{delivery_state}' -> Applied Shipping Fee: ${shipping_fee:.2f}")
```

3. End-to-End Applied Data Decisions Pipelines

Real-world data science classification, filtering, and imputation across student academic records and enterprise HR compensation datasets.

In [code]

```
# Pipeline 1: Academic Cohort Evaluation & Remediation Routing
student_cohort = [
    {"id": "S101", "name": "Aman Verma", "exam_score": 91, "attendance_pct": 88, "assignments_done":
True},
    {"id": "S102", "name": "Pooja Patel", "exam_score": 76, "attendance_pct": 92, "assignments_done":
True},
    {"id": "S103", "name": "Rohan Joshi", "exam_score": 54, "attendance_pct": 64, "assignments_done":
False},
    {"id": "S104", "name": "Sneha Sen", "exam_score": 82, "attendance_pct": 68, "assignments_done":
True},
    {"id": "S105", "name": "Vikas Rao", "exam_score": 44, "attendance_pct": 78, "assignments_done":
True}
]

print("=== STUDENT EVALUATION & COURSE COMPLETION REPORT ===")
for s in student_cohort:
    score = s["exam_score"]
    att = s["attendance_pct"]
    assign = s["assignments_done"]

    # Multi-condition decision tree
    if not assign:
        status = "Incomplete: Pending Assignments"
        recommendation = "Submit pending lab files to registrar"
    elif score >= 85 and att >= 80:
        status = "High Honors Distinction"
        recommendation = "Eligible for fast-track internship voucher"
    elif score >= 60 and att >= 70:
        status = "Cleared: Regular Pass"
        recommendation = "Proceed to next academic module"
    elif score >= 50 or att >= 75:
        status = "Probationary Pass"
        recommendation = "Attend 2-week mandatory review workshop"
    else:
        status = "Course Failed"
        recommendation = "Course retake required in upcoming term"

print(f"[{s['id']}] {s['name']:<12} | Score: {score:>2}% | Att: {att:>2}% | Result: {status}")
print(f"        Action Plan: {recommendation}")
```

In [code]

```
# Pipeline 2: Enterprise Compensation & Performance Appraisal Engine
employees = [
    {"emp_id": "E201", "name": "Aditi Roy", "dept": "Data Engineering", "salary": 125000, "rating":
4.8, "years": 4},
    {"emp_id": "E202", "name": "Karan Shah", "dept": "Support Ops", "salary": 44000, "rating": 3.4,
"years": 1},
    {"emp_id": "E203", "name": "Neha Gupta", "dept": "Data Science", "salary": 88000, "rating": 4.2,
"years": 3},
    {"emp_id": "E204", "name": "Rahul Nair", "dept": "Human Resources", "salary": 52000, "rating": 4.6,
"years": 5}
]

print("\n=== EMPLOYEE APPRAISAL & SALARY BAND DECISION REPORT ===")
for emp in employees:
    sal = emp["salary"]
    rating = emp["rating"]
    years = emp["years"]

    # 1. Salary Band Classification
    if sal >= 100000:
        tier = "Senior Executive"
    elif sal >= 60000:
        tier = "Mid-Level Professional"
    else:
        tier = "Associate Level"

    # 2. Performance Hike & Bonus Determination via Nested Logic
    if rating >= 4.5:
        if years >= 3:
            hike_pct = 15.0
            bonus_flag = "Top Performer Bonus ($5,000)"
        else:
            hike_pct = 12.0
            bonus_flag = "Spot Achievement Bonus ($2,500)"
    elif rating >= 3.8:
        hike_pct = 8.0
        bonus_flag = "Standard Appraisal"
    elif rating >= 3.0:
        hike_pct = 4.0
        bonus_flag = "Cost of Living Adjustment Only"
    else:
        hike_pct = 0.0
        bonus_flag = "Performance Improvement Plan (PIP)"

    new_salary = sal * (1 + hike_pct / 100)
    print(f"[{emp['emp_id']}] {emp['name']:<11} | Band: {tier:<20} | Hike: {hike_pct:>4.1f}% | New Sal:
${new_salary:,.2f}")
    print(f"        Status: {bonus_flag}")
```